

Nbody: retain state across all timesteps

The newest complete session reproduces the exact flat_pow improvement: median runtime falls from 227.19575 to 143.3736875 ms per 20,000-step unit, a 36.89% reduction (1.5846x). The previous full run showed 37.53% with the same source. This is confirmation under another collection block, not a new nbody optimization; grouped remains the earlier 3.40% intermediate.

Candidate / evidence	Median runtime reduction	Decision
grouped, earlier Ubuntu pair	3.40%	Useful intermediate; below 7%.
flat_pow, newest full Ubuntu pair	36.89% / 1.5846x	Exact default; reproduced previous 37.53%.
flat_sqrt, separate timing pair	38.58%	Optional approximate experiment.
New structural controls	Exact verification passed	Explain contributions; no extra speedup claimed.

The reference is the pyperformance five-body gravitational benchmark: Sun, Jupiter, Saturn, Uranus and Neptune. Each step processes ten unordered pairs, updates both bodies' velocities, then advances five positions. A unit has 20,000 steps at $dt = 0.01$: 200,000 pair updates and 100,000 position updates. Sixteen timed units accumulate state through 320,000 steps; they do not restart from the initial orbit.

BODIES maps names to tuples containing mutable position and velocity lists plus mass. SYSTEM and PAIRS alias those lists. Python floats are boxed objects. Dependencies are pyperf and standard Python; the final optimized software uses no external numeric library. All inputs and masses are read from supplied state; no trajectories or answers are precomputed.

Conceptually, for each pair $\delta = r_1 - r_2$ and $\text{mag} = dt * (\delta \cdot \delta) ** (-1.5)$. The two velocity increments are opposite, weighted by the other body's mass. Only after all ten pairs does each position advance by $dt * \text{velocity}$. Energy reporting sums kinetic energy and negative pair potential; momentum offset initializes the reference body before timing. The implementation preserves the reference's exact expression order, not just these real-number identities.

The wrapper loads fresh upstream state for each independent measurement and calls `upstream.bench_nbody`. Momentum offset occurs before its internal timer. Each timed unit runs energy reporting, advance, then energy reporting. The optimized wrapper substitutes only advance; validation, loading and writeback in that function remain timed. Energy reporting and the upstream loop remain unchanged.

Newest matched pair: `results/nbody_baseline_20260920-080806_16940` and `results/nbody_optimized_20260920-081744_16940`. Comparison: `results/session_all_20260920-074308_674/comparison_nbody`. Kernel `flat_pow` and all captured source hashes match current files.

Use independent instruments for different questions

The target is Ubuntu 22.04.5, Linux 5.15.0-1106-kvm, one virtual CPU exposing an Intel Xeon E5-2630 v3 at 2.40 GHz. Clean timing uses release CPython 3.10.12, CPU 0 affinity, a fixed hash seed and cyclic GC disabled. The VM governor is unavailable; host scheduling, frequency and contention are not fully controlled. These are measurements of this guest, not bare-metal Haswell limits.

Instrument	What it establishes	What it cannot establish
Clean timing	11 fresh processes per arm; upstream internal timer; every observation retained.	No proof against between-block drift.
perf stat	Separate repeated counting runs; instructions, cycles, branches and loads per equal work.	Its elapsed time is not the clean timing result.
perf record + FlameGraph	Native debug stacks, fixed 5-million-cycle sampling period; aggregate all folded stacks.	Not a release-time percentage or a count of Python calls.
cProfile	Exact call counts and Python function attribution in a separate run.	Tracing time is perturbed and is not a speedup measurement.
pyperformance / pyperf	Official suite baseline cross-check in its worker harness.	No official optimized-suite result is present.

The custom wrapper calls the actual upstream bench function and uses its timer. A minimal pyperf import stub exposes `time.perf_counter`; it does not implement `pyperf.Runner`. Thus this is an upstream-workload experiment through a custom harness, alongside a separate official pyperformance baseline run. That distinction does not invalidate the experiment; the workload, timer boundaries and comparison controls must be explicit.

Runtime reduction = $100 \times (1 - \text{optimized median} / \text{baseline median})$. Speedup = $\text{baseline median} / \text{optimized median}$. We use runtime reduction for the declared 7% target. CV = $\text{sample standard deviation} / \text{mean}$. Samples from separate TXT and CSV perf invocations are not pooled.

Container overhead was real, but not the whole cost

A flame graph is an aggregation of sampled call stacks, not a timeline. Width is inclusive sampled weight; height is stack depth, not elapsed time. Self weight counts only samples whose leaf is that symbol. Children weight includes descendants and overlaps other ancestors. The libc start routine can therefore have a huge inclusive share while doing little direct work. Aggregate a function across every folded stack: the top-30 listing is incomplete, and repeated appearances must not be mistaken for separate causes.

In the newest baseline native profile, PyObject_GetItem and PyObject_SetItem subtrees cover 206 of 1,175 samples (17.53%). Evaluator self weight is 526 (44.77%). Float operators, binary dispatch, boxing and release are also prominent. These categories overlap when inclusive; do not sum them into a partition of runtime.

The source explains the stack evidence. Every pair unpacks positions and masses; each `v[i]` update reads and writes a list element through Python's generic numeric/container machinery. A cache hit still needs interpreter instructions, bounds/index handling, reference updates and dispatch. Keeping a float reference in a local avoids those container operations even when the entire physical working set fits in cache.

Source-level work per timestep	Upstream	grouped	flat_pow
Logical component reads + writes	$150 + 75 = 225$	$114 + 57 = 171$	30 initial reads + 30 final writes per advance, outside the timestep loop
Pair traversal	10 entries / step	4 partner groups / step	10 explicit updates / step
Float-result expressions	270	270	270
Power expression	$dt * d2^{**}(-1.5)$	Same	Same

These are source-operation counts, not machine-load counts or runtime percentages. Entry shape validation and mass/container access are excluded from the component count. All float arithmetic remains: 5.4 million float-result expressions per unit. Locals still hold boxed floats, and interpreter freelists can reuse storage; neither an expression count nor a constructor call equals a fresh malloc.

CPython is a bytecode interpreter, not an AST interpreter. `_PyEval_EvalFrameDefault` includes bytecode execution, operand handling, loop control and reference work; its self share is not pure call overhead. Debug checks and different code generation change costs. High IPC and few cache misses support an instruction-work hypothesis but do not identify a precise hardware stall limit. Reject negative top-down fractions and unsupported PMU events rather than interpreting them as valid measurements or zeros.

Why grouped was modest; why flattening is different

grouped retains the first body's position and velocity across its consecutive partners, writes its velocity once per group, and updates the second body immediately. It preserves upstream pair and arithmetic order, including pow. It removes 24% of logical component accesses but still traverses groups, unpacks tuples and reloads state on every timestep. Four partner iterators per step add overhead. Retaining pow was intentional, not an experiment defect.

flat_pow loads all 30 position/velocity components into scalar locals once per advance, retains them through all timesteps, explicitly expands the ten pairs and five position updates in their original order, and writes state back once. It simultaneously removes most state subscripting, repeated unpacking and short-loop traversal. The outer timestep loop, actual physics, power expression and update ordering remain.

The specialization supports five bodies, canonical ordered pairs, distinct ordinary position/velocity lists of floats, float dt and nonnegative integer steps. Validation rejects unsupported shapes. It preserves list identity and successful-call state; delayed writeback does not preserve partial updates on exceptions, signal-time observations or arbitrary aliased/custom containers. This is an explicit benchmark contract, not universal Python API equivalence.

Candidate	What is changed	Status
sqrt	Replace inverse power by $dt / (d2 * \text{sqrt}(d2))$.	Approximate; not final default.
hoist / full	Legacy per-pair hoisting; full also uses sqrt.	Retained controls; full is not flat_pow.
flat_sqrt	All flat_pow structural changes plus sqrt rewrite.	Tolerance passes; bit identity fails.
Structural controls	Unpacked, indexed-grouped, unrolled-pairs/lists, velocity-locals and full-state controls.	Added in experiments/nbody_controls.py; exact checks pass.

A repeated small saving matters over 20,000 steps, but repetition multiplies both old and new work; it does not automatically multiply percentage speedup. The grouped and flat results show that the scope of state reuse, not merely the presence of local variables, determines the impact.

Correctness, alternatives and decisions

Acceptance for grouped and flat_pow requires finite, bit-identical energy and all 30 state components, checked as binary floating-point values. Energy alone is insufficient: different trajectories can share a similar total energy. Tests cover zero steps, small trajectories, randomized valid states/masses, list identity, invalid layout rejection and full accumulated batches.

The saved optimized gate in this new session compares the complete reference and candidate 16 x 20,000-step batch: energy -0.1690801979358367 and every state bit match for grouped and flat_pow. Earlier full results saved only a one-unit automatic gate; the new artifact closes that batch-evidence gap. The standalone baseline verifier still checks one unit despite a loops=16 phase label; the independent reference run inside the optimized gate provides the full comparison. The existing integrated software suite passed 63 tests; no benchmark code changed in this review.

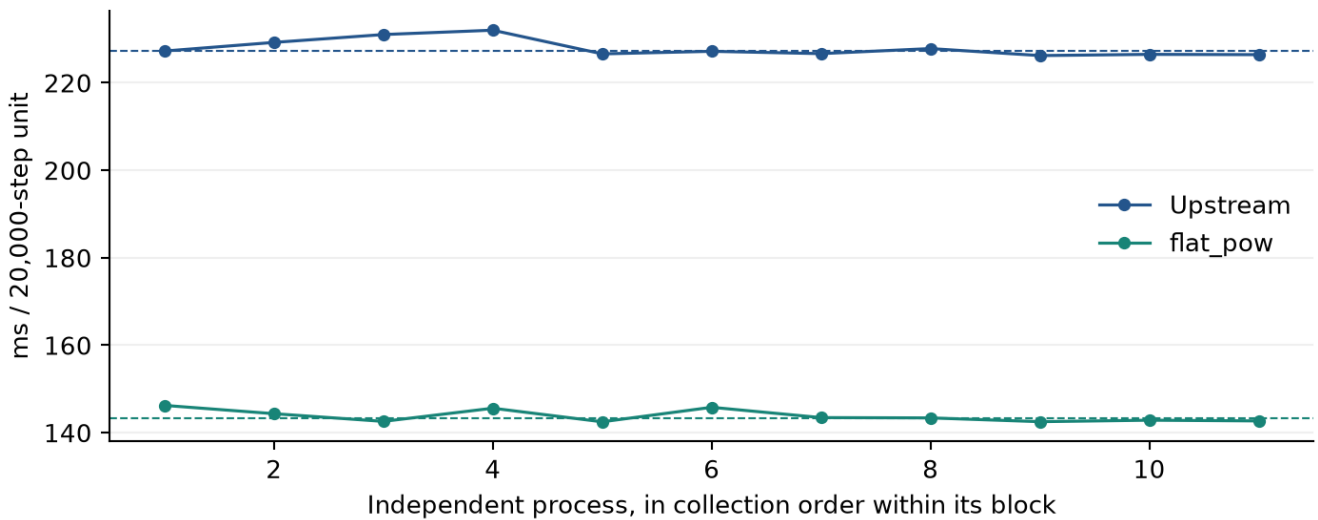
Idea	Reason / evidence	Decision
sqrt rewrite	At 16 units, relative energy error 2.364e-14; max state error 2.522e-10; normalized error 1.004e-11.	Retain as optional, reject as exact default.
Barnes-Hut / FMM	Only five bodies; approximation/tree overhead and different force evaluation.	Reject for this exact workload.
Fewer steps / new integrator	Changes the requested computation and trajectory.	Reject.
Plain NumPy tiny operations	Per-call overhead for five bodies; no measured benefit; reductions may reorder arithmetic.	Not implemented without evidence.
Compiled native backend	Could remove boxed interpretation cost; changes execution technology.	Deferred by agreed pure-Python scope.
More unrolling / expression reassociation	Timestep expansion increases code size; reassociation changes FP rounding.	No unverified addition to default.

The decision is to keep the measured exact flat_pow implementation and add diagnostic controls rather than claim that every proposed rewrite improves it. flat_sqrt contains the same flat/unrolled structure with a different arithmetic choice; there is no missing separate flat_unrolled kernel.

Contracts, commands and previous validation are retained in docs/NBODY_FLAT_EXPERIMENT.md, docs/NBODY_GROUPED_EXPERIMENT.md, docs/NBODY_CONTROL_EXPERIMENTS.md and tests/.

Newest full run confirms the previous improvement

Newest matched Ubuntu session: all clean observations



Eleven fresh processes per arm; 16 accumulated units per process. Every observation retained; arms collected in separate blocks.

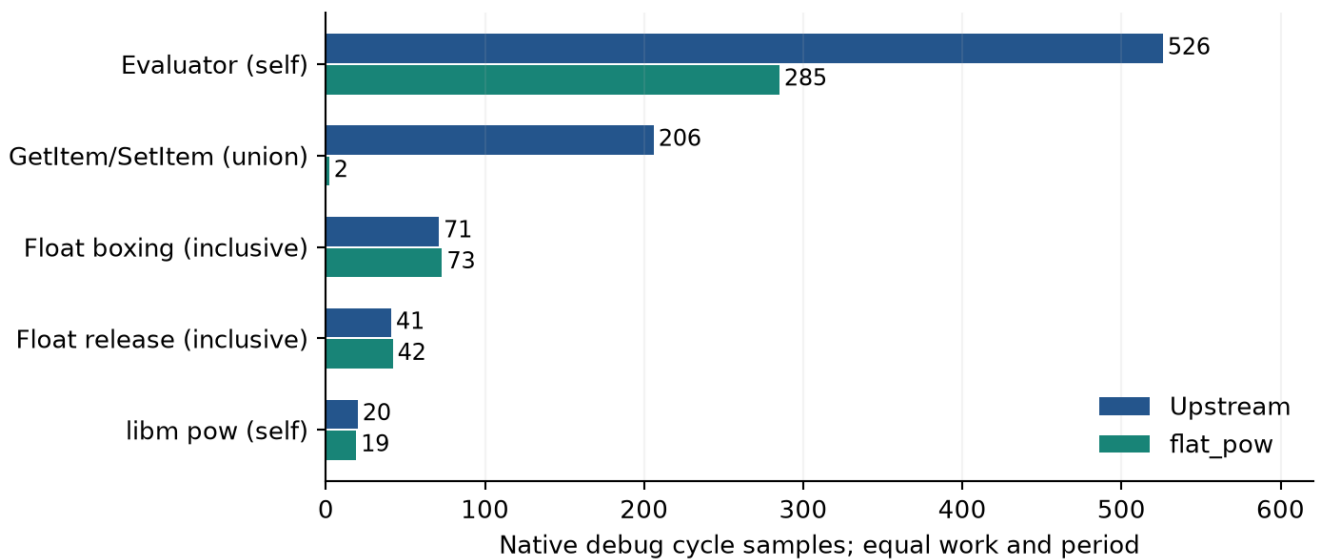
Clean timing per unit	Upstream	Optimized
Median	227.195750 ms	143.373688 ms
Mean +/- sample SD	227.906 +/- 1.990 ms	143.807 +/- 1.441 ms
CV	0.873%	1.002%
Minimum - maximum	226.222 - 232.022 ms	142.481 - 146.232 ms

Median runtime reduction is 36.8942% (1.5846x), saving 83.8221 ms per unit. Even the slowest optimized observation beats the fastest baseline by 35.36%. This exceeds the 7% runtime criterion with wide margin; within-block dispersion is not a bound on systematic VM drift.

The preceding full run was 228.24925 -> 142.578375 ms (37.53%). Relative to that run, this baseline is 0.46% faster and flat_pow is 0.56% slower. The source is identical; the 0.64 percentage-point change is a repeat-measurement difference, not evidence of an added optimization or a code regression. Grouped remains 3.40%; the optional flat_sqrt result remains a separate approximate experiment.

Latest official pyperformance baseline: 60 values, median 228.636 ms, mean 229.411 +/- 3.439 ms. It corroborates workload scale; no optimized official-harness result exists.

The removed work is reproduced in the new profiles



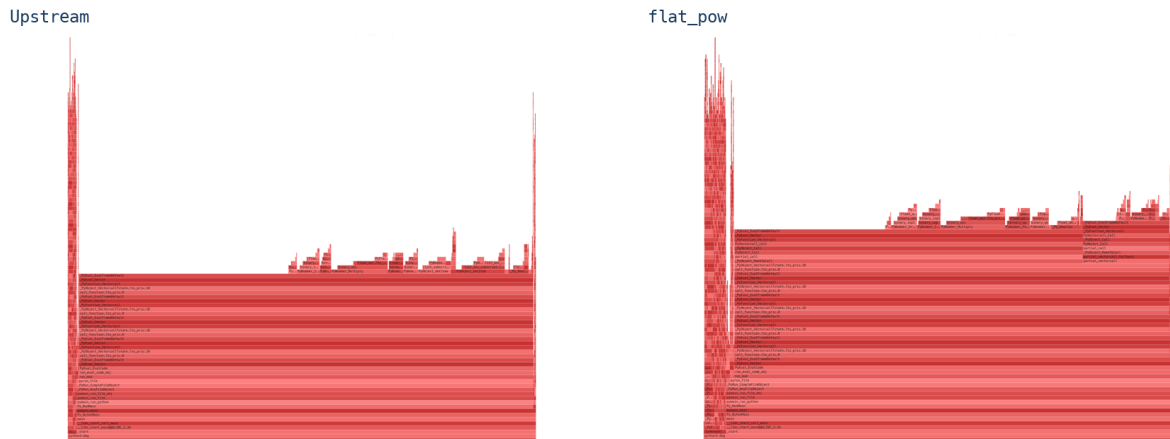
Equal native work: five units per arm, debug CPython, fixed 5-million-cycle period. Item access and float lifetime rows are inclusive unions; evaluator and libm pow are self. Do not sum rows.

Counter / unit (TXT)	Upstream	Optimized	Change
Instructions	1,691.786m	1,101.148m	-34.91%
Cycles	549.829m	347.974m	-36.71%
L1 data loads	535.846m	351.722m	-34.36%
Branches	193.593m	125.500m	-35.17%

Generic item-access union falls 206 -> 2 samples and evaluator self falls 526 -> 285. Total samples fall 1,175 -> 725. The preceding run showed the same structural change (226 -> 2 item-access samples). Independent CSV counting corroborates instructions -34.82%, cycles -37.03% and L1 loads -34.50%.

Float creation inclusive samples are 71 -> 73, release 41 -> 42, and libm pow self 20 -> 19. These noisy counts support arithmetic and boxed-value lifetime work remaining. Locals have not eliminated float-result objects or the power calculation. The gain comes mainly from reducing repeated state access, unpacking and interpreter traversal.

Read the similar stacks without missing the change



Newest original native SVG overviews. Each fills its own panel despite 1,175 versus 725 total samples. Adjacent original SVGs allow zooming.

Both runs still enter the same upstream timed loop, call advance and execute floating-point operations through CPython. Those common stack bases dominate the visual silhouette. The flat wrapper adds a different call path, so depth is not a direct work measure. Look for the missing subscription branches and compare equal-work leaf/subtree counts instead of only the outline.

Each cProfile arm still calls the numerical advance function once per unit. Whole-process call counts include optimized-wrapper startup and validation; they do not count arithmetic bytecodes as Python calls. Most remaining work is within the long advance call. Folded aggregation of all stacks is more reliable than comparing two partial top-30 listings.

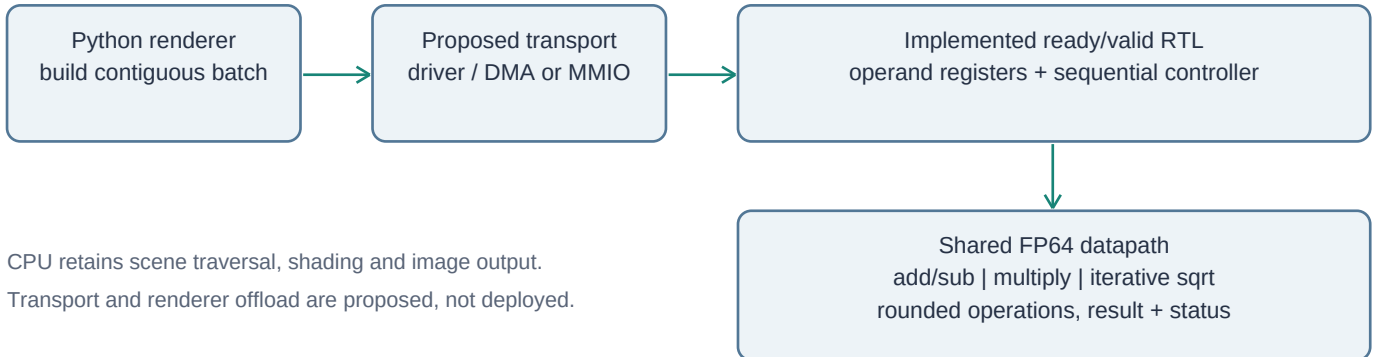
TXT and CSV counter batches agree on major instruction, cycle and load reductions. L1 misses disagree in direction between batches; generic cache events are noisy. No cache-miss improvement is established. Negative top-down fractions invalidate that decomposition and unsupported LLC events remain unavailable. These limitations do not invalidate the independent clean runtime result.

Session manifests capture the actual variant, upstream, wrapper, common/pipeline and configuration hashes; all match current files. Git metadata is disabled rather than guessed. The full saved optimized correctness gate covers reference and candidate at 16 units. However, there is no persisted phase-0 package authenticity transcript, and the embedded UNVERIFIED placeholder remains. Some phase rows stay requested despite present completed artifacts; py-spy is explicitly disabled.

The remaining bottleneck is a set of costs: arithmetic dispatch, boxed values, evaluator work and pow. An exact next software improvement must be tested against flat_pow. The new structural controls can distinguish locality from unrolling; the current aggregate speedup cannot uniquely allocate savings among them.

A small, explicit FP64 accelerator

One ray-sphere accelerator is implemented for the project. It is relevant to raytrace; a separate exact nbody engine is deferred.



Implemented accelerator boundary is the ready/valid RTL. The host transport is an integration proposal.

hardware/rtl/ray_sphere_accel.sv contains the sequential controller, operand/result registers, status and ready/valid protocol. The datapath uses actual unmodified BSD-licensed Berkeley HardFloat Release 1 RTL: one shared add/sub unit, one multiplier and an iterative square-root unit. There are no real-number or software-math placeholders in the synthesizable datapath.

```

cp = center - origin
v = (cp.x*dx + cp.y*dy) + cp.z*dz
q = (cp.x*cp.x + cp.y*cp.y) + cp.z*cp.z
disc = r*r - (q - v*v)
result = no_root if disc < 0 else v - sqrt(disc)
  
```

Each operation rounds separately to binary64, nearest-even, including gradual underflow. There is no FMA, reassociation, FP32 or approximate reciprocal root. A tangent and a negative returned distance preserve upstream behavior; software applies its original hit-selection rule. Finite inputs/intermediates, nonnegative radius and caller-normalized direction define the supported domain. Invalid/nonfinite cases return an error for CPU fallback.

HardFloat source and license: hardware/vendor/ provenance and license files. Primary specification: <https://www.jhauser.us/arithmetic/HardFloat-1/doc/HardFloat-Verilog.html>. Detailed ports, schedule and host format: docs/HARDWARE_DESIGN.md and hardware/.

Design reassessment: sphere intersections remain frequent, but square root is reached only 6,934 times (3.864% of 179,457 queries). The current core is a single-query sequential function with no resident scene or internal batch controller. A batched geometric core or a small reusable FP64 command engine are alternatives, not implemented replacements. More general hardware is not automatically faster; operand residency and the software offload boundary need justification.

Shared arithmetic reduces replication but constrains throughput; pipelining or multiple lanes costs area, registers and potential switching. No area/watt/achieved-clock numbers are claimed. Nbody offload still needs exact host-power semantics or an explicit different contract; this ray-sphere core does not accelerate nbody.

New workload evidence changes the estimate

Interface / validation	Current specification
Request / response	640 operand bits + 32-bit tag; distance/status/flags response; one outstanding query; ready/valid and held response.
Numerical contract	Separate FP64 rounding, ordered expressions, caller-normalized rays; finite supported domain with CPU fallback on error.
Implementation boundary	Arithmetic/control RTL and host packing exist. Renderer batching, transport/DMA and driver remain proposed.
Clock / schedule	50 MHz target assumption; 20 cycles/miss, 80/root budget. No achieved timing or HDL simulation.

The full-profile math.sqrt caller map records 6,934 sphere calls; 37,715 other square roots belong to magnitude and one to import setup. Thus $h = 6,934 / 179,457 = 0.0386388$. Source control flow confirms that sphere calls sqrt only after a nonnegative discriminant. This measured workload mix replaces the earlier all-root assumption; it is not a measured hardware speed.

With the same assumed 50 MHz, 1 GB/s aggregate payload bandwidth, batch size 1,024, 10 us launch/batch and 100 ns native packing/query, modeled core time is 80.1036 ms. Transfer is 18.6635 ms, launch 1.7600 ms and packing 17.9457 ms: 118.4728 ms total offload cost. Against the new full software time of 452.69625 ms, break-even requires removing more than 26.17% of release runtime.

Assumed removed fraction f	Estimated total time	Reduction vs full
20%	480.630 ms	-6.17% (slower)
30%	435.360 ms	3.83%
40%	390.091 ms	13.83%

$T_{\text{new}} = T_{\text{full}} \times (1 - f) + T_{\text{offload}}$. f remains unknown: cProfile sphere share cannot supply it. The modeled 7% runtime reduction would require $f > 33.17\%$; this is a sensitivity point, not a separate mandated hardware target. Batching can change early-exit queries and h, so the original query mix must be rechecked after integration.

The old all-root case against shadow-only software remains historical evidence, not the current default estimate. If applied to faster full software, that all-root stress case needs 71.90% removable time just to break even. No hardware improvement occurred between estimates; only software baseline and workload assumptions changed.

A successful exact optimization, with bounded claims

The new complete session confirms flat_pow at 36.89% lower runtime, following the previous 37.53% result with identical source. It retains the power expression and exact accumulated state. Grouped showed that some local reuse was insufficient; full-state locality and explicit pair/position updates remove much more repeated work, reproduced in both native profiles and independent counter batches.

A five-body hardware engine could hold state locally and execute complete advance batches, amortizing transfer across 20,000 steps. It is not the implemented accelerator: exact agreement with the host nonintegral pow routine, dependencies between velocity updates and full trajectory validation make it a different, larger design. A MAC/rsqrt replacement would change rounding and is unsuitable for the strict final contract without further proof and testing.

One concrete ray-sphere HDL design satisfies the chosen single-component hardware scope. Its architecture and losing as well as winning estimate scenarios are documented above. It does not speed up nbody, and syntax checks are not a claim of numerical correctness or achieved acceleration.

```
./run_all.sh
python3 -B variants/bm_nbody_upstream_opt.py \
--mode verify --loops 16 --iterations 20000
LOOPS=16 ./tools/ab_timing.sh nbody 16 --kernel flat_pow
python3 -B experiments/nbody_controls.py --mode verify \
--loops 16 --iterations 20000
```

Required deliverable	Location / evidence
Six-section TXT report	report_nbody.txt; this PDF is supplemental and uses the same source.
Execution scripts / README	script_nbody.sh, run_all.sh, tools/, README.md.
Variants and exact checks	variants/bm_nbody_upstream_opt.py; tests/; docs/evidence/.
Latest full evidence	Session074308_674: baseline080806_16940, optimized081744_16940; full directory names on page 1.
Analysis / hardware / AI disclosure	docs/reports/, hardware/, docs/HARDWARE_DESIGN.md, prompt.txt.
Remaining submission actions	Presentation and working demo; author-controlled repository upload; optional VM raytrace ablations and refined hardware integration.

Project.pdf pp. 3-5 and 7-8 drive these sections; docs/PROJECT_REQUIREMENTS.md maps each requirement. No Git operations were performed. New measurements require deliberate source/report updates; the pipeline will not overwrite authored interpretation.